Here is a comprehensive end-semester examination study guide covering **Control Statements**, **Dictionaries**, **Functions**, and **Object-Oriented Programming (OOP)**, grounded strictly in your study material sources.

---

1. Control Statements

Control statements determine the flow of execution in a Python program, allowing the interpreter to make decisions, execute loops, or break sequence based on evaluated conditions.

A. Decision-Making Constructs (`if` , `elif` , `else`)

By default, Python executes code sequentially from top to bottom. The `if` statement allows Python to select which block of code to run based on boolean evaluation (`True` or `False`).

- **Syntax Rules**: Every conditional header must end with a colon (`:`), and the body block must be indented (4 spaces or 1 tab).
- **Execution Flow**: In an `if / elif / else` ladder, conditions are tested sequentially from top to bottom. As soon as one condition evaluates to `True` , its corresponding block runs, and the remainder of the ladder is skipped. The `else` block executes only if all preceding conditions fail.

```
marks = 78

if marks >= 90:
    print("Grade: O")
elif marks >= 80:
    print("Grade: A+")
elif marks >= 70:
    print("Grade: A")  # Executes because 78 >= 70
else:
    print("Grade: B")
```

B. Iteration Constructs (`for` and `while` Loops)

- **`for` Loop**: Used when the number of iterations is known in advance or when iterating over sequence collections (lists, tuples, strings, ranges).

```
for i in range(1, 6):  # Generates numbers 1 to 5 (stop boundary is exclusive)
    print(i, end=" ")
```

- **`while` Loop**: Repeats a code block continuously as long as a boolean condition remains `True`.
    - **Infinite Loop Warning**: If the condition never changes to `False`, the loop runs indefinitely. Some internal variable within the loop body must update the condition, or a `break` statement must be executed.

```
count = 1
while count <= 5:
    print("Count:", count)
    count += 1  # State modification to prevent infinite looping
```

C. Loop Control Keywords (`break`, `continue`, `pass`)

- **`break`**: Immediately terminates the innermost loop entirely and jumps out to the next statement outside the loop.
- **`continue`**: Interrupts the current iteration, skips all remaining statements inside the loop body for that cycle, and jumps directly to the next iteration.
- **`pass`**: A null placeholder statement that does nothing; used to satisfy Python's syntactic requirement for an indented block where code is planned for later.

---

2. Dictionaries

A dictionary is a mutable, ordered (Python 3.7+) collection that stores data as key-value pairs (`{key: value}`).

A. Rules Governing Keys and Values

- **Keys**: Must be **unique** and **immutable** (strings, integers, floats, or tuples; lists can **never** be used as keys). If duplicate keys are declared, the last value overwrites all earlier ones.
- **Values**: Can be any data type, including mutable sequences like lists or nested dictionaries.

B. Essential Dictionary Operations & Methods

- **Direct & Safe Access**: Accessing `dict[key]` returns the corresponding value but raises a `KeyError` if the key does not exist. Using `.get(key, default)` returns `None` or a custom default value without crashing the program.
- **Inspection Views**:
  - `.keys()` : Returns a view object of all dictionary keys.
  - `.values()` : Returns a view object of all stored values.
  - `.items()` : Returns a view object of all `(key, value)` tuple pairs, making dictionary iteration in `for` loops straightforward.
- **Modification & Merging**:
  - `.update(dict2)` : Merges another dictionary in-place.
  - **Merge Operator (`|`)**: Combines two dictionaries into a new dictionary instance (Python 3.9+).
  - **Dictionary Unpacking (`**`)**: Unpacks key-value pairs into a new dictionary structure.
- **Deletion**: `.pop(key)` removes the specified key and returns its value. The `del dict[key]` statement deletes the key directly.

```
student = {"name": "Alpha", "roll": 101, "marks":}

# Safe access
print(student.get("age", "N/A"))  # Returns 'N/A' instead of KeyError

# Merging dictionaries using unpacking
marks = {"python": 95, "math": 90}
combined = {**student, **marks}  # Combines both dictionaries
```

---

3. Functions

A function is a reusable block of code that executes only when explicitly invoked. Functions promote modularity and prevent code duplication (the DRY principle).

A. Parameters vs. Arguments

- **Parameters**: Variable names defined in the function header inside parentheses.
- **Arguments**: The actual data values passed into the function during a call.

B. Argument Passing Conventions

1. **Positional Arguments**: Arguments are matched to parameters based on their position/order.

2. **Keyword Arguments**: Arguments are passed with explicit parameter identifiers (`param=value`), allowing them to be supplied in any order.
3. **Arbitrary Positional Arguments (`*args`)**: Packs any number of extra positional arguments into a single **tuple** parameter inside the function.
4. **Arbitrary Keyword Arguments (`**kwargs`)**: Packs varying keyword arguments into a single **dictionary** parameter.

```
# Function using *args and **kwargs
def process_data(*args, **kwargs):
    for num in args:
        print("Arg:", num)
    for key, value in kwargs.items():
        print(f"{key}: {value}")

process_data(10, 20, name="Alpha", score=95)
```

C. Return Values & Anonymous Lambda Functions

- **`return` Statement**: Passes a computed result back to the caller. Functions without an explicit `return` return `None` by default.
- **Lambda Functions**: One-line anonymous inline functions defined using the `lambda` keyword. Syntax: `lambda parameters: expression`.

```
# Standard function vs Lambda equivalent
square_lambda = lambda x: x * x
print(square_lambda(5))  # Output: 25
```

---

4. Object-Oriented Programming (OOP)

Object-Oriented Programming structures code around **Classes** (blueprints/templates) and **Objects** (physical instances created from those blueprints containing attributes and methods).

A. The `__init__` Constructor and `self`

- **`__init__` Method**: A special dunder (double underscore) method called automatically by Python when an object is instantiated to set up initial instance attributes.

- **`self` Parameter**: Refers to the specific instance currently invoking the method. It must be the first parameter in all instance methods, but Python passes it automatically during calls.

```
class Dog:
    def __init__(self, name, age):
        self.name = name  # Instance attribute unique to each object
        self.age = age

    def bark(self):  # Instance method
        print(f"{self.name} says Woof!")

buddy = Dog("Buddy", 3)  # Instantiation
buddy.bark()  # Output: Buddy says Woof!
```

B. Variable and Method Scopes

- **Instance Variables vs. Class Variables**:
  - **Instance Variables**: Declared inside `__init__` using `self.var`; unique to each individual object.
  - **Class Variables**: Declared inside the class body but outside methods; shared by all instances of that class.
- **Class Methods vs. Static Methods**:
  - **`@classmethod`**: Receives `cls` (the class itself) as its first parameter; can inspect or modify class state.
  - **`@staticmethod`**: Takes no `self` or `cls` parameter; acts as an isolated utility function placed inside the class for logical grouping.

C. Inheritance, Method Overriding, and `super()`

- **Inheritance**: Allows a child class (`subclass`) to inherit all attributes and methods from a parent class (`superclass`), enabling code reuse (`class Child(Parent):`).
- **Method Overriding**: Occurs when a child class redefines an inherited method using the exact same name to customize its behavior.
- **`super()` Function**: Returns a temporary object representing the parent class, allowing the child to invoke parent methods—most commonly `super().__init__()`—to extend setup logic rather than rewriting it.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
```

```
        self.salary = salary

class Manager(Employee):  # Inherits from Employee
    def __init__(self, name, salary, department):
        super().__init__(name, salary)  # Reuses parent constructor
        self.department = department

    def display(self):  # Overridden or custom method
        print(f"{self.name} manages {self.department}")
```

D. Dunder Methods & Operator Overloading

Dunder methods allow custom objects to integrate with Python's built-in syntax and operators:

- **`__str__(self)`** : Defines a clean human-readable string representation called when `print(obj)` or `str(obj)` is executed.
- **`__add__(self, other)`** : Overloads the addition (`+`) operator for custom class instances.

---

5. Exam Practice Question (20 Marks)

Question (20 Marks total):

Design a continuous Python application for a **Student Registration and Fee Management System** combining OOP, Dictionaries, Functions, Exception Handling, and a Tkinter GUI.

- **(a) [5 Marks]**: Define a custom exception `InvalidFeeError` that inherits from `Exception` and takes the invalid fee amount in its constructor.
- **(b) [10 Marks]**: Create an OOP model `StudentRegistry` that uses a dictionary to store student records (`roll_no` as key, dictionary of details as value). Implement methods to register students and calculate total fees collected, raising `InvalidFeeError` if fee is negative.
- **© [5 Marks]**: Build a Tkinter GUI interface using `Frame` , `Entry` , `Listbox` , and `messagebox` widgets that accepts input, validates formats using `try/except` , displays enrolled students in a `Listbox` , and handles errors gracefully.

Solution 1: Baseline Implementation (Simple Solution)

```
# (a) Custom Exception
class InvalidFeeError(Exception):
    def __init__(self, fee):
        super().__init__(f"Fee cannot be negative: {fee}")

# (b) Registry Class using Dictionaries
class StudentRegistry:
    def __init__(self):
        self.students = {}  # Dictionary to store roll -> details

    def register(self, roll, name, fee):
        if fee < 0:
            raise InvalidFeeError(fee)
        self.students[roll] = {"name": name, "fee": fee}

# Driver code
registry = StudentRegistry()
try:
    registry.register(101, "Alpha", 5000)
    registry.register(102, "Beta", -1000)  # Triggers exception
except InvalidFeeError as e:
    print("Error:", e)
```

Solution 2: Complex Improvement (Full 20/20 Marks — Differentiable Solution)

This complete, continuous solution integrates modular functions, dictionary storage, OOP encapsulation, custom exception validation, and a Tkinter GUI frontend.

```
import tkinter as tk
from tkinter import messagebox

# --- (a) Custom Exception Class ---
class InvalidFeeError(Exception):
    """Raised when an invalid or negative fee amount is supplied."""
    def __init__(self, fee_amount):
        self.fee_amount = fee_amount
        self.message = f"Invalid Fee: ${fee_amount:.2f}. Fee must be a positive value
        super().__init__(self.message)

# --- (b) OOP Model Class Using Dictionaries ---
class StudentRegistry:
    def __init__(self):
```

```
        # Dictionary structure: {roll_no: {"name": str, "fee": float}}
        self._database = {}

    def add_student(self, roll_no, name, fee):
        """Validates parameters and inserts student record into dictionary."""
        if roll_no in self._database:
            raise KeyError(f"Student with Roll No {roll_no} is already registered.")

        if fee < 0:
            raise InvalidFeeError(fee)

        # Store in nested dictionary
        self._database[roll_no] = {"name": name, "fee": fee}

    def get_all_students(self):
        """Returns dictionary items for GUI display."""
        return self._database.items()

    def calculate_total_fees(self):
        """Computes aggregate fees using dictionary iteration."""
        return sum(record["fee"] for record in self._database.values())

# --- (c) Tkinter GUI Application ---
class StudentRegistrationApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Student Fee Registration System")
        self.geometry("480x400")
        self.config(bg="#f4f6f7")

        self.registry = StudentRegistry()
        self.build_ui()

    def build_ui(self):
        # Header Panel
        header = tk.Frame(self, bg="#2c3e50", pady=10)
        header.pack(side=tk.TOP, fill=tk.X)
        tk.Label(header, text="Student Registration Studio", font=("Arial", 14, "bold

        # Input Form (Grid Layout inside Frame)
        form_frame = tk.LabelFrame(self, text=" Registration Form ", font=("Arial", 1
        form_frame.pack(pady=10, fill=tk.X, padx=15)

        tk.Label(form_frame, text="Roll No:", bg="#f4f6f7").grid(row=0, column=0, sti
        self.ent_roll = tk.Entry(form_frame, width=15)
        self.ent_roll.grid(row=0, column=1, pady=3, padx=5)
```

```
        tk.Label(form_frame, text="Student Name:", bg="#f4f6f7").grid(row=1, column=0
        self.ent_name = tk.Entry(form_frame, width=15)
        self.ent_name.grid(row=1, column=1, pady=3, padx=5)

        tk.Label(form_frame, text="Fee ($):", bg="#f4f6f7").grid(row=2, column=0, sti
        self.ent_fee = tk.Entry(form_frame, width=15)
        self.ent_fee.grid(row=2, column=1, pady=3, padx=5)

        btn_register = tk.Button(form_frame, text="Register", command=self.handle_reg
        btn_register.grid(row=0, column=2, rowspan=3, padx=15, sticky="nesw")

        # Listbox Display Area
        list_frame = tk.Frame(self, bg="#f4f6f7")
        list_frame.pack(pady=5, fill=tk.BOTH, expand=True, padx=15)

        self.scrollbar = tk.Scrollbar(list_frame, orient=tk.VERTICAL)
        self.listbox = tk.Listbox(list_frame, font=("Courier", 10), yscrollcommand=se
        self.scrollbar.config(command=self.listbox.yview)

        self.listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        # Footer Action Button
        footer = tk.Frame(self, bg="#f4f6f7")
        footer.pack(side=tk.BOTTOM, fill=tk.X, pady=10, padx=15)

        tk.Button(footer, text="Show Revenue Summary", command=self.show_summary, bg=

    def handle_registration(self):
        """Extracts input, validates formats, and handles custom exceptions."""
        try:
            roll_str = self.ent_roll.get().strip()
            name = self.ent_name.get().strip()
            fee_str = self.ent_fee.get().strip()

            if not roll_str or not name or not fee_str:
                raise ValueError("All fields are mandatory.")

            roll_no = int(roll_str)
            fee = float(fee_str)

            # Register student in model
            self.registry.add_student(roll_no, name, fee)

            # Update Listbox UI
```

```
            display_str = f"Roll: {roll_no:<5} | Name: {name:<12} | Fee: ${fee:.2f}"
            self.listbox.insert(tk.END, display_str)

            # Reset Input Form
            self.ent_roll.delete(0, tk.END)
            self.ent_name.delete(0, tk.END)
            self.ent_fee.delete(0, tk.END)
            self.ent_roll.focus()

        except ValueError as ex:
            messagebox.showerror("Input Format Error", f"Validation Failed: {ex}")
        except KeyError as ex:
            messagebox.showwarning("Duplicate Entry", str(ex))
        except InvalidFeeError as ex:
            messagebox.showwarning("Fee Boundary Error", str(ex))

    def show_summary(self):
        """Displays total aggregated fees collected."""
        total = self.registry.calculate_total_fees()
        count = len(list(self.registry.get_all_students()))
        messagebox.showinfo("Revenue Summary", f"Total Registered Students: {count}\r

if __name__ == "__main__":
    app = StudentRegistrationApp()
    app.mainloop()
```

💡 **Next Steps**: Would you like to practice tracing output for tricky loop/control-flow edge cases, or explore database integration with `sqlite3` for GUI applications next?